

Proyecto Final de Machine Learning para Astronomía

Javiera Vivanco, Javiera Rojas, Sofía Saavedra
Alexander Stolzenbach

4 de diciembre de 2024

Objetivo

El objetivo principal del proyecto es construir un modelo de Machine Learning que clasifique diferentes tipos de estrellas en función de sus propiedades físicas, como temperatura, luminosidad, radio, magnitud absoluta, color y clase espectral. Este modelo utiliza el conjunto de datos "Star Dataset" de Kaggle.

Selección de Datos

Se analiza un set de datos con 240 estrellas y 7 features, temperatura absoluta, luminosidad relativa, radio relativo, magnitud absoluta, color de estrella, clase de espectro, tipo de estrella. Estos features tienen datos teóricos y empíricos, es decir, que de los datos observados, se obtuvieron los datos faltantes usando ecuaciones y leyes de la física. Las ecuaciones que se usaron fueron Ley de Stefan Boltzmann (para la luminosidad) Ley de Wienn (para encontrar temperatura) relación de la magnitud absoluta y paralaje (para el radio). Usando estas features, podremos clasificar el tipo de estrella, Brown Dwarf, Red Dwarf, White Dwarf, Main Sequence, Supergiant, Hypergiant. Para poder hacer más fácil la clasificación de estrellas, y de paso, más rápida.

Exploración de Datos y Preprocesamiento

Revisando los datos nos dimos cuenta que no hay valores faltantes ni outliers. Las variables color de estrella y clase de espectro son strings es por ello que al momento de usar un algoritmo se transforma a número. Las visualizaciones que usaremos para observar la relación entre los features y la variable objetivo (tipo de estrella) serán histogramas.

Selección de Algoritmos

Se escoge kNN y RF, ya que contamos con pocos feature y con una relativamente baja cantidad de datos, el algoritmo de RF lograra encontrar bisecciones pertinentes a cada variable (temperatura, luminosidad), y kNN se ajustará a cada grupo basandose en los vecinos cercanos, por lo que podrá reconocer los de Main Sequence.

Entrenamiento y Evaluación

Random Forest (RF)

Para evaluar el modelo de Random Forest, generamos la matriz de confusión, que se presenta en la Figura 2. Además, calculamos distintas métricas de evaluación, tales como la Accuracy, Recall y Precision, donde TP representa los verdaderos positivos, TN los verdaderos negativos, FP los falsos positivos, y FN los falsos negativos. A continuación, se muestran los valores obtenidos para estas métricas, donde los valores de Recall y Precision corresponden a cada una de las categorías, desde la 0 hasta la 5.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}. \quad (1)$$

Accuracy: 0.99; **Recall por clase:** [1.00, 1.00, 1.00, 1.00, 0.95, 1.00]; **Precision por clase:** [1.00, 1.00, 1.00, 1.00, 1.00, 0.94].

Esto significa que el desempeño del modelo Random Forest es excelente y casi perfecto. Sin embargo, en la quinta clase, correspondiente a Hypergiant, la precisión es menor a 1, lo que indica que una estrella fue clasificada erróneamente como parte de esta clase (falso positivo). Además, como el recall de la cuarta categoría, Supergiant, es menor a 1, esto sugiere que la estrella mal clasificada como clase 5 en realidad pertenece a la clase 4. Todas las demás clases tienen todos sus verdaderos positivos correctamente clasificados como verdaderos positivos, lo que indica un desempeño perfecto en términos de recall y precisión para esas categorías.

Para estudiar el bias y la varianza en el modelo, calculamos la métrica Accuracy tanto para las predicciones realizadas en el conjunto de entrenamiento como para las predicciones en el conjunto de prueba. Los resultados obtenidos fueron un Train Accuracy de 1.00 y un Test Accuracy de 0.99. Un valor de 1.00 en Train Accuracy indica que nuestro modelo predice perfectamente el conjunto de entrenamiento, lo que significa que el modelo ha aprendido correctamente de esos datos. Por otro lado, un valor de 0.99 en Test Accuracy es también bastante alto, y significa que el modelo generaliza bien y no está haciendo sobreajuste (overfitting) a los datos de entrenamiento. Esto implica que el modelo clasifica muy bien los casos que no conocía durante el entrenamiento. Estos valores indican un bajo bias y una baja varianza en el modelo, lo cual es ideal, ya que predicará muy bien los tipos de estrellas de nuevas estrellas, para las cuales no conozcamos su etiqueta.

K-nearest neighbors (KNN)

En este caso, también creamos una matriz de confusión, mostrada en la Figura, y calculamos las métricas de Accuracy, Recall y Precision. Los puntajes obtenidos para estas métricas son los siguientes:

Accuracy: 0.97; **Recall por clase:** [1.00, 1.00, 1.00, 1.00, 0.87, 0.92]; **Precision por clase:** [1.00, 1.00, 1.00, 0.89, 0.87, 1.00].

A partir de estos resultados, se puede decir que el modelo KNN tiene muy buen desempeño, logrando identificar correctamente todos los verdaderos positivos para las categorías 0 (Brown Dwarf), 1 (Red Dwarf) y 2 (White Dwarf). Sin embargo, para las clases 4 (Supergiant) y 5 (Hypergiant), el recall es menor a 1, debido a la presencia de falsos negativos, lo que implica que algunas estrellas de estas categorías fueron clasificadas incorrectamente en categorías adyacentes. Por otro lado, para las categorías 3 y 4, el precision es menor a 1 ya que presentan falsos positivos, es decir, estrellas de otras clases fueron clasificadas como clase 3 o 4. Aun así, los valores de las métricas se mantienen altos, lo que indica que el modelo KNN es un bueno.

Replicamos el procedimiento anterior para analizar el bias y la varianza del modelo, calculando la métrica Accuracy tanto para las predicciones realizadas en el conjunto de entrenamiento como en el conjunto de prueba. Para el modelo KNN, obtuvimos un Train Accuracy de 0.99 y un Test Accuracy de 0.97. Como se mencionó anteriormente, un valor alto de train accuracy indica que el modelo aprendió correctamente a partir de los datos de entrenamiento, mientras que un valor alto de test accuracy demuestra que el modelo es capaz de clasificar correctamente estrellas nuevas, es decir, aquellas que no formaron parte del entrenamiento. Esto sugiere que el modelo no sufrió de overfitting. Por lo tanto, nuestro modelo presenta un bajo bias y una baja varianza.

Conclusiones

En este caso, debido a que el problema es de clasificación de estrellas con múltiples características físicas, consideramos que Random Forest es más adecuado. Esto se debe a su robustez frente al ruido y su menor sensibilidad a la escala de los datos, además, con este método se obtuvo una precisión de 0.99.

Uno de los cambios que se podría hacer si tuviéramos acceso a más datos sería probar un modelo de redes neuronales. Por otro lado, un experimento adicional interesante que se podría hacer, es analizar la importancia de las características, por ejemplo, implementar feature_importance en Random Forest.

Bibliografía

- [1] Kaggle: *Star Dataset*. Disponible en: <https://www.kaggle.com/deepu1109/star-dataset>.

A. Apéndice

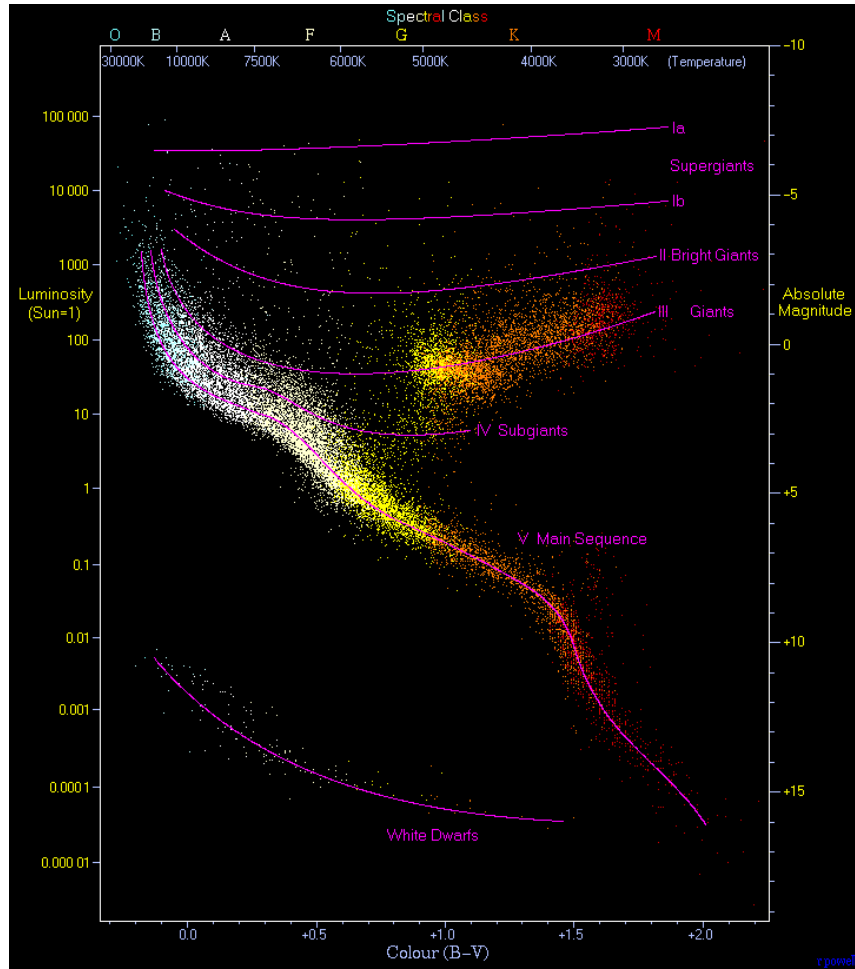


Figura 1: Diagrama Hertzsprung-Russell, clasifica estrellas basado en su luminosidad y temperatura

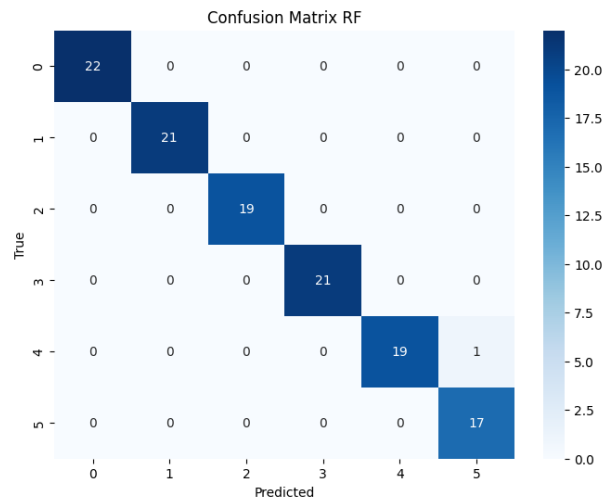


Figura 2: Matriz de confusión del modelo RF.

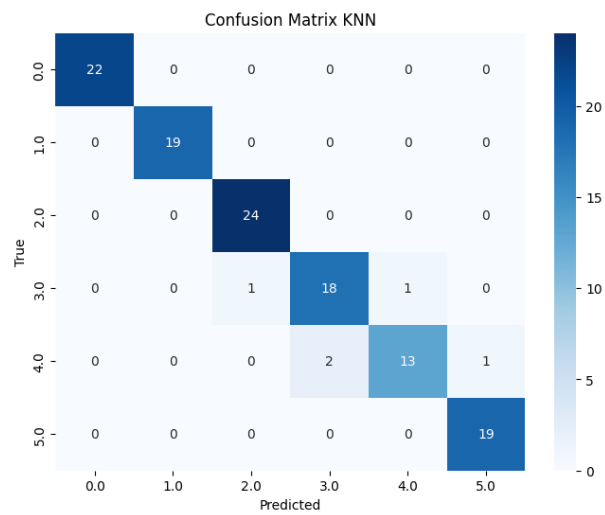


Figura 3: Matriz de confusión del modelo KNN.